Can you suggest a simple but powerful project using imagejev for my ai engineer portfolio.Don't write code now.Just the project plan.

A simple but portfolio-strong project is an **AI Visual QA & Triage Agent for Web Apps**: it watches screenshots from a web application, uses JEV-27B-VL to decide whether each screen is correct, classify any issue, and route it to the right team. It is practical, demo-friendly, and shows real AI-engineering skills: vision, decision APIs, confidence thresholds, automation, and evaluation. JEV is well suited because it returns calibrated probabilities for yes/no, choice, and rating decisions over images and text.[1]

Project: "Visual QA Triage Agent"

One-line pitch:
Upload or automatically capture screenshots of a web app; the agent decides whether the UI looks correct, identifies the likely issue, assigns severity, and routes it to the appropriate team.

Why this project works

- It uses JEV's strengths: image understanding plus structured decisions, not open-ended chat.[1]
- It solves a believable business problem: UI regressions, broken layouts, missing buttons, and incorrect states.
- It is easy to demo in a portfolio with before/after screenshots and a dashboard.
- It can be extended later into browser automation, UI regression testing, or a computer-use agent.

Core workflow

```
Screenshot / UI state
        ↓
Preprocessing & metadata
        ↓
JEV-27B-VL decision calls
        ↓
Confidence threshold + rules
        ↓
Issue report / dashboard / Slack or email alert
```

The agent can run on screenshots uploaded by a user, screenshots generated by Playwright during tests, or images submitted through a simple web UI.

What JEV decides

Use separate, focused decision calls rather than one vague prompt:

Decision	JEV output
“Does this screen look visually correct?”	Yes / No, with probability
“What type of issue is shown?”	Layout break, missing element, text overflow, broken image, empty state, loading error, none
“How severe is this issue?”	0–5 rating
“Which team should handle it?”	Frontend, backend, design, content, QA
“Is this a duplicate of an existing issue?”	Yes / No

This maps directly to JEV’s supported decision types: yes/no, selection among 2–256 options, and 0–5 scoring. [1]

Features to build

1. Screenshot intake

- Upload screenshots manually.
- Capture pages automatically with Playwright.
- Store image, URL, page name, timestamp, browser, and viewport size.

2. AI decision engine

- Send each screenshot to JEV with a clear question and limited options.
- Require a confidence threshold, for example 0.70.
- If confidence is low, mark the case as “needs human review” instead of forcing a wrong classification.

3. Issue dashboard

A small web dashboard showing:

- Screenshot preview
- Detected issue type
- Severity
- Assigned team
- Confidence score

- Status: open, reviewing, resolved
- History of similar screenshots

4. Duplicate detection

Before creating a new issue, compare it with recent issues using:

- Same page/URL
- Similar screenshot
- Similar JEV issue classification

Then ask JEV: “Is this new screenshot the same issue as the selected previous issue?” This prevents dozens of duplicate tickets.

5. Notifications

Send high-severity or low-confidence issues to Slack, email, or a ticketing system. Keep the first version simple: a dashboard plus optional Slack webhook.

Demo scenario

Use a small demo e-commerce site with intentionally injected bugs:

- Broken product image
- Misaligned checkout button
- Empty shopping-cart page
- Text overflowing on mobile
- Missing price on a product card
- Incorrect error message after payment failure

Your portfolio demo can show the agent receiving a screenshot and producing:

```
Issue: Missing price on product card
Severity: 4/5
Team: Frontend
Confidence: 0.88
Recommended action: Create ticket
```

Evaluation plan

Build a labeled test set of 50–100 screenshots:

- Correct screens
- Known UI bugs
- Duplicate cases
- Ambiguous cases

Measure:

- Classification accuracy
- Yes/no detection precision and recall
- Severity-rating agreement
- Duplicate-detection accuracy
- Percentage sent to human review
- End-to-end latency

This evaluation section will make the project look much stronger than a simple demo.

Architecture

Component	Purpose
Frontend	Upload screenshots and view issues
Backend API	Manage screenshots, decisions, and issue records
JEV service	Visual decision engine via `/v1/decide`
Playwright runner	Automatically capture app screenshots
Database	Store screenshots, decisions, and issue history
Dashboard	Review, approve, and resolve issues
Notification service	Alert teams for high-priority issues

Portfolio presentation

Include these in your GitHub README or case study:

- Problem statement
- Architecture diagram
- Screenshots of the dashboard
- Example JEV decision inputs and outputs
- Evaluation metrics
- Limitations: hallucination risk, ambiguous screenshots, need for human review
- Future work: browser-agent mode, automatic Playwright regression testing, multi-page flow validation

My opinion

I'd choose this over the virtual-park robotics project for a first portfolio project because it is **faster to build, easier to demo, and more immediately understandable to recruiters**. The robotics project is impressive but requires simulator, robotics, and control-system knowledge; the Visual QA Triage Agent delivers a complete end-to-end AI product with less infrastructure.

A strong future extension is to turn it into a **self-healing UI test agent**: Playwright captures a failing screenshot, JEV identifies the issue and next UI action, and the system retries or creates a detailed bug report. That would demonstrate both visual AI and agentic automation. JEV is explicitly positioned for judging whether an agent completed a task from its trajectory and screenshot, so this extension fits naturally. [1]

⁂

1. https://mixpeek.com/model/autotrust/JEV-27B-VL
2. https://huggingface.co/aioutfitters/JEV-27B-VL-AWQ/blame/main/README.md
3. https://huggingface.co/aioutfitters/JEV-27B-VL-AWQ/blob/44392aa19ff846f19a30b74137d501664862a3c5/README.md
4. https://www.prnewswire.com/apac/news-releases/autotrust-ais-jev-27b-vl-tops-hugging-face-global-trending-list-as-open-decision-models-gain-momentum-302902296.html
5. https://thinkplainai.com/blog/jev-27b-vl
6. https://huggingface.co/aioutfitters/JEV-27B-VL-AWQ/commit/3396d0d7681bd2de48a2e129126365fd4ac64e91
7. https://www.prnewswire.com/news-releases/autotrust-ais-jev-27b-vl-tops-hugging-face-global-trending-list-as-open-decision-models-gain-momentum-302902279.html
8. https://huggingface.co/autotrust/JEV-27B-VL/raw/main/README.md
9. https://huggingface.co/autotrust/JEV-27B-VL/blob/main/README.md
10. https://huggingface.co/autotrust/JEV-27B-VL/blob/main/blog/JEV-27B-VL.md
11. https://huggingface.co/aioutfitters/JEV-27B-VL-AWQ/blob/main/README.md
12. https://autotrust.ai/blog/jev-27b-vl-inference-shell
13. https://huggingface.co/blog/autotrust/autotrustjev-27b-vl-a-decision-model-that-learned
14. https://huggingface.co/autotrust/JEV-27B-VL/blob/4000d2393be6718e604f8e7dca563a780ab78e78/README.md
15. https://huggingface.co/autotrust/JEV-27B-VL